Conceptual Architecture of WezTerm

CISC 322 Assignment 1: Conceptual Architecture Report

[WezTerm Warriors / 18]

Ben van Mil (23ljmh@queensu.ca) Miles Chant (24wfb2@queensu.ca)
Owen Macey (owen.macey@queensu.ca)
Eric Charest (eric.charest@queensu.ca) Abraham Li Wu (22rmf2@queensu.ca)

October 2026

Abstract

This report presents the conceptual architecture of WezTerm, an open-source, GPU-accelerated terminal emulator and multiplexer written in Rust by Wez Furlong. It is intended to help a developer or manager who is new to the project understand what the major parts of the system are, why they exist, and how they work together, based on WezTerm's documentation and the organization of its source repository.

WezTerm is organized into six subsystems: the GUI / Frontend Layer, the GPU Rendering Pipeline, the Multiplexer (Mux), the Terminal Emulation Core, the Configuration System, and SSH / Networking. The Multiplexer is the hub: it owns every window, tab and pane, delegates the creation of panes to local or remote domains, and notifies the GUI whenever something changes. The Terminal Emulation Core is a self-contained state machine that turns a shell's output into a grid of cells, while the Configuration System, driven by a user-written Lua program, is a cross-cutting dependency read by every other subsystem.

The overall style is layered, combined with a component-based decomposition into Rust crates, event-driven notifications from the Mux to the GUI, and a pipeline on the output path. Concurrency follows a hybrid model of a main GUI thread, an asynchronous executor, and background threads for each pane. These choices are driven by five objectives: GPU-accelerated rendering, cross-platform support, Lua extensibility, built-in multiplexing and remote sessions, and low latency. Two use cases, typing a key and opening a new tab, are traced through the architecture with sequence diagrams. We conclude that the architecture favours responsiveness, portability and evolvability, at the cost of a larger codebase to maintain, dependence on graphics drivers, and a Lua API that must remain stable for users.

1 Introduction and Overview

This report describes the conceptual architecture of WezTerm: the major subsystems that make up the system, the responsibilities of each, and how they interact to deliver its functionality. It concentrates on goals, requirements and interactions rather than on individual classes or functions. The architecture was recovered from WezTerm's documentation [1] and the structure of its source repository [2].

1.1 What Is WezTerm?

WezTerm is a GPU-accelerated cross-platform terminal emulator and multiplexer implemented in Rust. It runs on Linux, macOS, Windows, FreeBSD, and NetBSD [1].

Essentially, a terminal emulator has two functions. It starts a program, such as a shell, and reads the text that program prints. That text may contain hidden instructions called escape sequences, which tell the terminal to perform actions like clearing the screen or changing colours [3]. The terminal follows these instructions, keeps track of what each character on the screen looks like, and then displays the result in a window. In the other direction, it sends the user's keystrokes back to the program.

WezTerm adds three main features on top of this basic job. First is GPU acceleration: it uses the computer's graphics card to draw text, which keeps it very fast and smooth. Second is a built-in multiplexer: it can manage many terminal sessions at once, in tabs and split panes, and keeps them active

so users can access them later or from other devices [4]. Finally, it has a Lua configuration program, meaning its users can customize it to a far greater extent than a typical terminal would allow [5].

1.2 History

WezTerm was created by Wez Furlong as a personal project. Its code was first published to GitHub in February 2018 [2]. WezTerm is an open-source project under the MIT license, meaning anyone can use, change and share the code for free. Furlong is still the main maintainer, but many outside contributors have added features and fixes over the years. By October 2026, WezTerm had over 29,000 GitHub stars.

Instead of using version numbers like 1.0 and 2.0, WezTerm releases are named with the date and time they were built (e.g., 20220904-064125-9a6cee2b) [6]. The project has also moved over time: it now has its own WezTerm organization on GitHub, and the documentation has moved to its own website, wezterm.org.

As WezTerm grew, the codebase expanded into a Cargo workspace of several smaller crates instead of a single large program. Some of these crates are designed so they can be reused in other projects: *wezterm-term* (the emulator core), *termwiz* (terminal utilities) [7], and *portable-pty* (cross-platform pseudo-terminals) [8].

1.3 Report Organization and Key Findings

Section 2 describes WezTerm's stakeholders and the objectives, constraints, quality requirements and risks that shape its design. Section 3 presents the conceptual architecture: the six subsystems and their dependencies, the architectural styles they combine, concurrency, evolution, and the division of developer responsibilities. Section 4 lists the external interfaces, and Section 5 traces two use cases, typing a key and opening a new tab, through the architecture, including the data that flows between subsystems. Sections 6 to 9 give a data dictionary, naming conventions, conclusions and lessons learnt. Our main findings are:

- **The Multiplexer is the centre of the system.** The GUI talks only to the Mux, which reaches shells through domains, so local and remote panes look the same.
- **Input and output are decoupled.** A keypress goes to the shell; the screen changes only when the shell's output returns and the Mux notifies the GUI.
- **The Terminal Emulation Core is isolated.** It knows nothing about windows, GPUs or networks, which makes it testable and reusable as a library.
- **Configuration is a program.** Nearly every subsystem reads the user's Lua configuration, so its API must stay backward compatible.
- **The design trades size for quality.** A larger, mostly single-maintainer codebase buys responsiveness, portability and evolvability.

2 Architecture Requirements

2.1 Stakeholders

Several groups of people depend on WezTerm, and they want contrasting things. For example, users typically ask for new features, but plugin authors want existing features to stay the same to avoid fixing all their plugins whenever WezTerm gets updated. Balancing the needs of many stakeholders is what shapes WezTerm's updates and design choices.

- **End users** are mostly programmers, developers and system administrators who use a terminal every day. They want speedy, correct text rendering, support for things like emojis and ligatures, tabs and split panes, and a built-in SSH client without needing to install extra tools, all using a single config across machines. These needs drive the low-latency and cross-platform objectives in Section 2.2.

- **Configuration and plugin authors** write Lua code to customize WezTerm. They need the Lua features to be well documented and to keep working the same way between versions to avoid breakage after an update. Because configuration is its own subsystem, the rest of WezTerm can change without breaking users' files [5].
- **Contributors and maintainers** write and fix WezTerm's code. They split the project into separate pieces that interact with each other in limited ways, so a change or update in one area does not accidentally break another.
- **Packagers** prepare WezTerm for app stores and Linux distributions, such as Homebrew, Flatpak and Winget. They prefer the program to build reliably and depend on as few outside libraries as possible.
- **Other developers** sometimes reuse individual WezTerm crates in their own projects, such as *portable-pty* or the Terminal Emulation Core, which is published as a separate library [9]. They need these crates to work without the rest of WezTerm.

2.2 Key Objectives

WezTerm's design is driven by five main objectives, each of which produces one or more of its six subsystems (Table 1).

Table 1. Key objectives and the subsystems they shape.

Key objective	Primary subsystem	Also shapes
GPU-accelerated rendering	GPU Rendering Pipeline	GUI / Frontend Layer
Cross-platform support	GUI / Frontend Layer	GPU Rendering Pipeline, SSH / Networking
Lua extensibility	Configuration System	GUI / Frontend Layer, Multiplexer
Built-in multiplexing and remote sessions	Multiplexer	SSH / Networking, Terminal Emulation Core
Low latency	Multiplexer, Terminal Emulation Core	GPU Rendering Pipeline, GUI / Frontend Layer

GPU-accelerated rendering: Rather than relying on the operating system's built-in text drawing, WezTerm takes advantage of the computational power of a GPU and uses it to draw the terminal. It supports two GPU front-ends, OpenGL (default) and WebGpu, as well as a CPU fallback (called Software in the config) [10]. This allows for better performance in areas where typical CPU-based text drawing might struggle, such as quickly scrolling output from a log. This objective is why the GPU Rendering Pipeline is a separate subsystem in WezTerm's architecture.

Cross-platform support: WezTerm runs on Linux, macOS, and Windows 10 and later [1] from one shared codebase [2]. The GUI / Frontend Layer keeps most platform-specific windowing and input code behind a common interface [2]. This way, the Multiplexer, Terminal Emulation Core, and renderer can expect the same input and behave the same on every OS. Since text is shaped and rasterized the same way on every platform (using HarfBuzz and FreeType [11], [12]), WezTerm looks consistent across systems. Building a separate native application for each platform could integrate more closely with each OS, but would require maintaining multiple codebases instead of one.

Lua extensibility: WezTerm's configuration is managed by a user-written Lua file rather than a fixed settings file [5]. This makes the Configuration System a subsystem that others can query for information at runtime. For example, the GUI Layer checks the Configuration System for key bindings before a keystroke reaches the shell (Figure 2). A plain settings file would be simpler and easier to check for mistakes, but it can only store fixed values. A Lua file allows users to write logic and have their configuration react to events from other subsystems.

Built-in multiplexing and remote sessions: Some terminal emulators, such as Alacritty, rely on additional tools such as tmux to manage tabs, panes, and sessions [13]. WezTerm, however, handles this itself and has its own SSH client [4], [14]. This objective is why WezTerm has a Multiplexer and its domain abstraction, which lets a pane run locally or on a remote machine without the rest of the system knowing the difference. Using tmux would shrink the codebase, but panes could not be drawn natively by the GUI.

Low latency: For the system not to feel slow and clunky, the delay between a keypress and the updated screen must be minimal. Unlike the other objectives, this spans several subsystems. The Terminal Emulation Core reads the shell's output in the background so it doesn't freeze the window. The Multiplexer groups changes together and tells the GUI when they are ready, so the screen is redrawn once for all of them. The renderer caches text that has already been drawn rather than drawing it again every frame [2]. Latency therefore matters in every subsystem.

2.3 Constraints

WezTerm had to be designed around several fixed constraints. It is written in Rust, a language that will not compile code that could let two threads interfere with each other's data [15]. This is why background work, such as reading shell output, has to pass its results to the GUI in a controlled way rather than changing the screen directly (Section 3.4). WezTerm's other constraints explain decisions described in Section 2.2: operating systems handle windows and text differently, which led to the shared windowing layer and text libraries [2], [11], [12], and graphics drivers vary between machines, which led to the CPU fallback [10].

2.4 Non-Functional Requirements

Responsiveness and portability are covered in Section 2.2. Table 2 lists WezTerm's other quality requirements, the design decision behind each, and its trade-off.

Table 2. Non-functional requirements, the design decisions that support them, and their trade-offs.

Requirement	Design decision	Trade-off
Rendering quality	A dedicated text-shaping step handles ligatures and emoji [11]	Relies on outside libraries
Unicode and right-to-left text	The Terminal Emulation Core uses a dedicated library for mixed-direction text [2]	More work for the core
Reliability	Sessions belong to the Multiplexer rather than the window, so they can outlive it when using a multiplexer domain [4]	The Multiplexer and GUI must stay coordinated
Evolvability	Lua configuration and separate subsystems let features be added without rewriting others	The Lua API becomes a long-term promise to users
Testability	The Terminal Emulation Core does not depend on the GUI, windowing, or the Multiplexer [2]	Relies on other subsystems for input and drawing

2.5 Risks

Several of WezTerm's design decisions carry risks. Building in a multiplexer and SSH client means more code to maintain, which is difficult in a project developed largely by one person [2]. Relying on the GPU ties how well WezTerm draws to each machine's graphics drivers; the CPU-based option reduces this risk but does not remove it [10]. Finally, offering users a Lua configuration API [5] means every option must keep working in future versions, which makes the Configuration System harder to change over time.

3 Conceptual Architecture

3.1 Major Subsystems

WezTerm is divided into six subsystems. Table 3 maps each one to the Rust crates that implement it, and the paragraphs that follow describe their responsibilities.

Table 3. Mapping of WezTerm's Rust crates to its six subsystems.

Subsystem	Primary crates
Terminal Emulation Core	wezterm-term, wezterm-escape-parser, vtparse, termwiz, wezterm-cell
Multiplexer	mux, portable-pty, wezterm-mux-server
GUI / Frontend Layer	wezterm-gui, window, wezterm-gui-subcommands
GPU Rendering Pipeline	wezterm-gui (rendering modules), wezterm-font
Configuration System	config, luahelper, lua-api-crates
SSH / Networking	wezterm-ssh, wezterm-client, codec, wezterm-mux-server-impl

Terminal Emulation Core (TEC). The TEC is classified as a pure state machine, meaning subsequent states and outputs rely only on the current state and the input. It is given bytes from a pseudo-terminal (PTY), parses escape sequences [3] and updates a Screen, a grid of cells each assigned a character and attributes like colour, bold, italic or hyperlink, as well as a scrollback buffer. Because it is deliberately kept separate from the rest of the system, it has no knowledge of GPUs, windows or networks and can be used as a standalone library.

Multiplexer (Mux). The Mux is the hub of the system. It keeps track of every terminal session that is open. Sessions are organized like folders: a window holds tabs, and a tab holds one or more panes, where each pane is a single running program such as a shell [16]. When the user requests a new pane, the Mux offloads the job to a domain, which represents where the program will run: on the local computer, on a remote machine through Secure Shell (SSH), inside WSL (Windows Subsystem for Linux), or on a separate WezTerm server [4]. Whenever something changes, such as new output or a pane closing, the Mux announces it and the GUI reacts accordingly.

GUI / Frontend Layer. The GUI is the part of WezTerm that the user actually sees and interacts with. The *window* crate acts as a translator: Windows, macOS and Linux each create windows and report keystrokes in their own way (through Win32, Cocoa, X11 or Wayland), and this crate hides those differences from the rest of WezTerm so it functions the same on every system. The *wezterm-gui* crate is built on it and runs the terminal application itself. When the user gives an input, such as pressing a key or clicking the mouse, it checks the user's key bindings: a shortcut such as "open a new tab" is sent to the Mux, whereas normal typing is passed to the program running in the current pane. It also draws the parts around the terminal text, like the tab bar, command palette and search overlays, and listens for the Mux's announcements to update the display.

GPU Rendering Pipeline. This subsystem converts the terminal's text into the displayed image. It uses the graphics card (GPU) to do the drawing, because a GPU can draw thousands of small images at once much faster than the main processor. It starts with the *wezterm-font* crate finding the right font [17] and working out how the characters should be arranged using a library called HarfBuzz, which handles ligatures, emoji and complex non-Latin scripts [11]. Another library, FreeType, rasterizes glyphs, which are then stored in an atlas, a large image kept on the graphics card, so they can be reused anywhere on screen. Users can pick how the drawing is done with the *front_end* setting: OpenGL (default), WebGPU via *wgpu*, or Software, which uses the main processor instead of the GPU (typically a fallback) [10].

Configuration System. This subsystem dictates how WezTerm appears and behaves, such as fonts, colours and key bindings. Most programs use a plain settings file that only lists options and values, whereas WezTerm uses a small program written in Lua, a lightweight scripting language, saved as

wezterm.lua [5]. Because the configuration is a program, the options are far more diverse: users can use logic, create keyboard shortcuts, and run code when certain events happen. WezTerm has a Lua interpreter built in; when it starts, it runs the user's file and converts the result into settings that the rest of the program can use. It also watches the file for changes, so it can apply new settings immediately without a restart (hot reloading). Almost every other subsystem reads from the configuration, so it is deeply interlinked with the inner workings of WezTerm.

SSH / Networking. This subsystem lets WezTerm connect to other computers, not just the one it is running on. WezTerm has its own built-in SSH client, *wezterm-ssh* [14]. SSH is the standard way to log on securely to a remote computer; typically, a user runs a separate SSH program inside the terminal, but WezTerm can do this itself. A remote device can then act as a domain, so the user can open tabs and panes on it as if it were their own computer. WezTerm can also split itself into two parts: a server that keeps the sessions running, and a client that connects to it. This allows a user to disconnect and reconnect later to find their programs still running [4]. The two parts communicate using a shared message format defined by the *codec* crate, with *wezterm-mux-server-impl* on the server side and *wezterm-client* on the client side [18].

3.2 Subsystem Interactions and Dependencies

Figure 1 shows the six major subsystems of WezTerm and how they depend on one another. For example, both the GUI and the GPU Rendering Pipeline point to the Multiplexer because they both need it to manage the terminal sessions they display. The Configuration System sits at the bottom and has arrows coming in from every other subsystem, showing its cross-cutting dependency. Information moves between the subsystems in both directions: program output flows up from the shell through the Terminal Emulation Core and the Mux to the GUI and renderer, while keystrokes and actions flow down from the GUI through the Mux to the program in the pane. Section 5 traces both directions in detail.

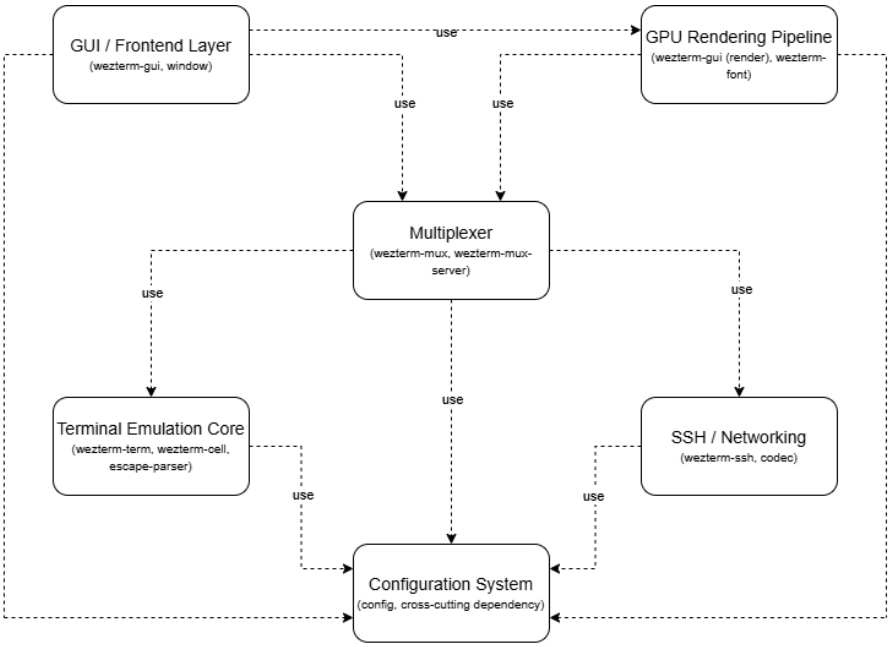


Figure 1. Dependency diagram of WezTerm's six major subsystems. Legend: each box is a subsystem, with its main crates in parentheses; a dashed arrow labelled "use" from A to B means that A depends on B (calls it or reads its data).

3.3 Architectural Styles

WezTerm combines several architectural styles, in the sense of Garlan and Shaw [19].

Layered architecture. WezTerm is structured in layers centred on the Multiplexer, where higher-level parts of the system depend on lower-level ones. At the top, the GUI and rendering pipeline handle everything the user sees and interacts with directly. Below that is the Multiplexer, which manages terminal sessions and passes data between the frontend and the underlying terminal logic, and below the Mux the Terminal Emulation Core keeps track of what each program has written to the screen. Each layer only communicates with adjacent layers; for example, the GUI never deals directly with the terminal core, and on the input path the GUI Layer calls the Multiplexer, which calls the Terminal Emulation Core. The Configuration System and SSH / Networking are not layers of their own: configuration is a cross-cutting foundation that every other part reads from, and SSH lets the Mux run programs on other computers. Layering keeps different concerns separated; for instance, a change to how configuration is stored does not require changes to the rendering pipeline.

Component-based decomposition. Rather than being built as one large program, WezTerm is broken up into subsystems that each fulfil a defined role. The Terminal Emulation Core handles escape sequences and screen state, the SSH subsystem handles networking, and the GPU Rendering Pipeline handles text rendering, with each one able to be understood and changed independently of the others. This structure also directly reflects how the Rust workspace is organized, with each subsystem mapping to one or more separate crates that can be compiled and tested on their own (Table 3).

Event-driven communication. Rather than subsystems calling each other directly on the output path, WezTerm uses an event and notification model. The Multiplexer notifies the GUI layer when pane content changes, and the GUI subscribes rather than polling. This decoupling means the Terminal Emulation Core and GPU Rendering Pipeline can operate independently. The Lua-based Configuration System uses this pattern as well: configuration changes trigger reload events that propagate to all subsystems without requiring tight coupling to the configuration loading mechanism.

Pipeline. The output path moves in stages: raw shell output, parsed escape sequences, updated terminal state, shaped glyphs and finally a drawn frame (Section 5.1).

3.4 Concurrency

Concurrency lets a program work on several tasks at once, and WezTerm must stay responsive while several shells are running. It uses an async Rust threading model, which is a hybrid between strict multithreading and asynchronous task management. The model has a main thread that runs the GUI event loop, an asynchronous executor to handle non-blocking tasks, and, for each pane, dedicated background threads that read and parse its shell's output [20]. These components communicate mostly through message passing and callbacks, with shared state limited to the Multiplexer's registries (Section 5.3), meaning a pane that is producing heavy output should not block either input handling or redrawing.

Because components communicate through events (Section 3.3), they only have to interact when an event is triggered. WezTerm can also use a multiplexer (mux) server, which follows a client-server model in which the GUI client and the mux server communicate to manage terminal sessions across local, SSH and mux-server domains [4], allowing many panes to run concurrently while WezTerm remains responsive.

WezTerm's concurrency model does have constraints, specifically for Lua synchronous event handlers, which execute on the main GUI thread and must return quickly to avoid blocking the interface [21]. When operations run for a long time on the GUI thread, responsiveness is reduced; this can be solved by delegating time-consuming work to background tasks. Separating tasks brings potential risks, such as race conditions, where a result depends on timing or order, and deadlocks, where two threads each hold something the other needs, leaving both stuck.

3.5 System Evolution

All systems need to be able to adapt to new requirements and add new content over time; this attribute is a system's evolvability. WezTerm uses a modular architecture so that single components can be modified

without affecting the others, making it much simpler to add or modify features. Its Lua-based Configuration System, with hot reloading, lets users customize fonts, colours, keyboard shortcuts and behaviour without changing the core components of the system.

Lua also allows users to write scripts that change WezTerm's behaviour, including event handlers that respond to events with custom actions [22]. These functions are important for evolvability because they let users add their own functionality to WezTerm without having to change or understand all of the code that makes it up. WezTerm also hides operating-system differences behind abstraction layers (Section 3.1), so, for example, changing how one operating system's windows are handled does not require rewriting the terminal parser.

Overall, WezTerm's modular architecture, which separates all the main components, makes each component easier to work with individually. New functionality, such as additional mux domains, can be added through existing interfaces without changes to major components. However, changing a shared interface can affect the components that depend on it, so compatibility must be considered as the system evolves.

3.6 Division of Developer Responsibilities

WezTerm's modular architecture allows developers to specialize in specific parts of the program without needing to understand the others in full. For example, a developer working on the GUI subsystem can focus solely on user input and window management, while a developer working on the Terminal Emulation Core focuses on parsing terminal output and maintaining terminal state. Table 4 shows the resulting areas of ownership.

Table 4. Areas of ownership and typical contributor focus.

Area	Subsystems	Typical contributor focus
Input handling, windows, key bindings	GUI Layer, Configuration	Platform integration, user-facing behaviour
Panes, tabs, sessions, domains	Multiplexer, SSH / Networking	Session management, remote connectivity
Escape sequences and screen state	Terminal Emulation Core	Protocol compatibility
Drawing	GPU Rendering Pipeline	Graphics and font rendering

Collaboration is still required when modifications change the way subsystems interact through shared interfaces. Testing is also streamlined with the modular architecture approach as each subsystem can be tested individually much easier though testing all together is still necessary. Overall, the modular architecture in WezTerm make development and maintenance much simpler while still requiring some collaboration between developers.

4 External Interfaces

WezTerm exchanges the following information with its environment:

- **User input and display (GUI / Frontend Layer).** Keyboard, mouse and window events arrive from the operating system's windowing system (Win32, Cocoa, X11 or Wayland); rendered frames are presented through OpenGL, WebGPU or, in Software mode, the CPU [10]. Copy and paste actions exchange text with the system clipboard.
- **Shell processes (Multiplexer, via domains).** For local panes, the command line, environment and terminal size are passed to a new process on a pseudo-terminal; encoded keystrokes are written to it and output bytes are read back.
- **Configuration file (Configuration System).** The user's *wezterm.lua* file is read at start-up and re-read when it changes; it supplies settings, key bindings and Lua event handlers [5].

- **Fonts (GPU Rendering Pipeline).** Font files installed on the system are located, then shaped and rasterized through HarfBuzz and FreeType [11], [12].
- **Remote hosts (SSH / Networking).** SSH sessions to remote machines, and framed binary mux-protocol messages exchanged with a WezTerm mux server over SSH, TLS or a Unix socket [4], [18].

5 Use Cases

5.1 Use Case 1: Typing a Key

A user types a character into the WezTerm window and sees the result on the screen. This is the most frequent interaction with the system, and users expect each character to appear instantly. Together with opening a new tab (Section 5.2), it demonstrates how the six subsystems cooperate: the GUI Layer, Configuration, the Multiplexer, the Terminal Emulation Core, SSH / Networking, and the GPU Rendering Pipeline (Figure 2).

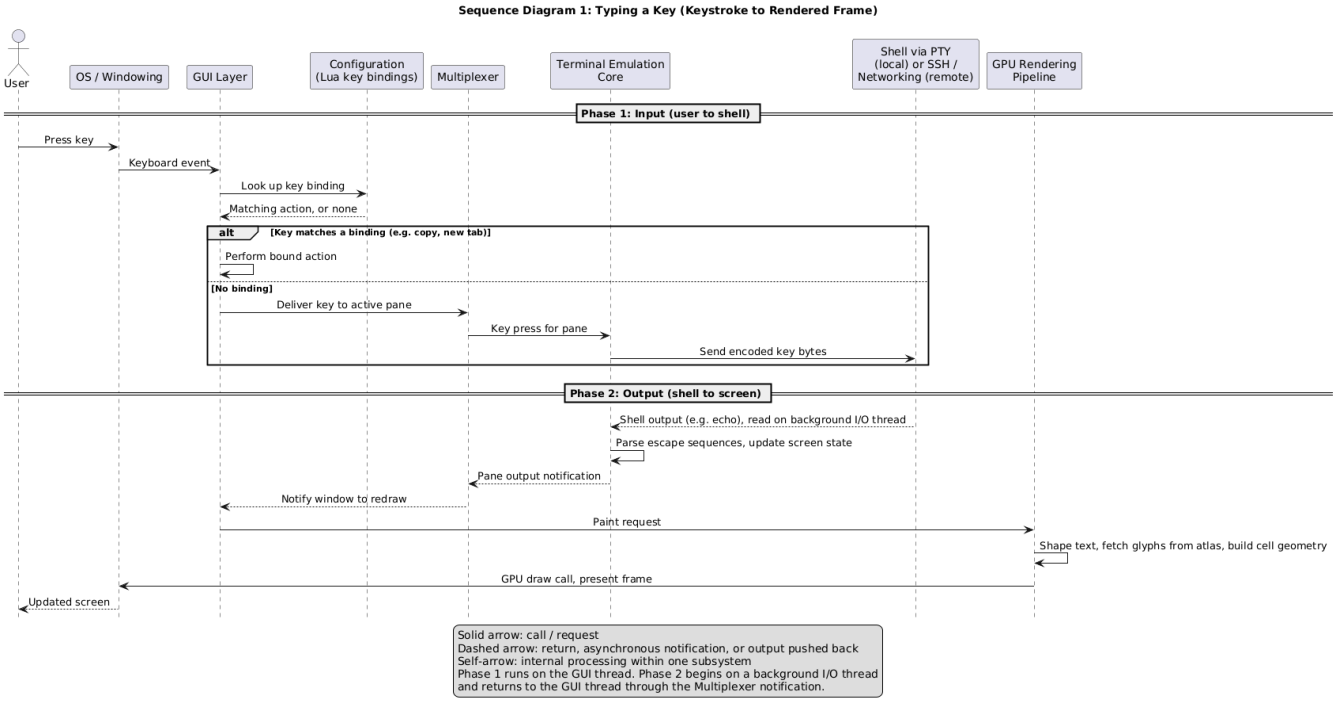


Figure 2. Sequence diagram for the keystroke use case. Legend: solid arrow = call/request; dashed arrow = return or asynchronous notification; self-arrow = internal processing within one subsystem.

Key insight: input and output are decoupled. WezTerm is a terminal emulator: it creates a pseudo-terminal (PTY), spawns a shell inside it, sends the user's input to the shell and interprets the shell's output [8]. A keypress therefore does not draw a character by itself. The key is sent to the pane's shell, and the shell's reply (usually an echo of the character) is what changes the screen. The use case has two phases linked by the shell connection:

1. **Input phase.** The operating system delivers a keyboard event to the GUI Layer. The GUI Layer consults the Configuration System, which holds the user's Lua key bindings, so users can redefine what keys do. If the key is bound to a WezTerm action (copy, paste, new tab), WezTerm handles it and the shell never sees the key. Otherwise, the GUI Layer delivers the key to the active pane through the Multiplexer. The Terminal Emulation Core encodes the key and sends it to the pane's shell.
2. **Output phase.** The shell's output returns over the same connection and is read on a background thread, so the window does not freeze while waiting for it. The Terminal Emulation Core parses escape sequences and updates the terminal state (cells, cursor, scrollback). The Multiplexer notifies the GUI

Layer that the pane changed. The GUI Layer requests a repaint, and the GPU Rendering Pipeline shapes the text, fetches glyphs from its glyph atlas and draws the frame.

Remote panes. The flow is identical for a pane running on a remote machine. The Multiplexer treats every pane as belonging to a domain. For a local domain, the shell connection is a local PTY; for a remote domain, it is carried by the SSH / Networking subsystem. Only the shell connection differs, so the GUI, Terminal Emulation Core and renderer do not need to know where the shell runs.

Global control flow. WezTerm is event-driven: control passes between subsystems through events and notifications, not through one sequential call chain. Table 5 shows who initiates each handoff.

Table 5. Control handoffs in the keystroke use case (TEC = Terminal Emulation Core).

#	From	To	Initiated by	How control passes
1	OS	GUI Layer	OS (user input)	Event arrives at the window's event loop
2	GUI Layer	Configuration	GUI Layer	Key-binding lookup
3	GUI Layer	Multiplexer	GUI Layer	Call (key not bound)
4	Multiplexer	TEC	Multiplexer	Call to active pane
5	TEC	Shell / SSH	TEC	Encoded key sent to shell connection
6	Shell / SSH	TEC	Shell	Output read on a background I/O thread
7	TEC	Multiplexer	TEC	Pane output notification
8	Multiplexer	GUI Layer	Multiplexer	Notification to subscribers
9	GUI Layer	GPU Rendering Pipeline	GUI Layer	Paint request on the GUI thread
10	GPU Rendering Pipeline	OS / GPU	Renderer	Draw call and frame presentation

Who is in control. The GUI Layer owns the main event loop and starts all user-driven control (steps 1–5, 9 and 10). The shell starts the other path: its output arrives asynchronously (step 6) and drives steps 7–10 with no further user action. The Multiplexer is the intermediary that decouples the GUI from the individual panes. The renderer never initiates control; it only responds to paint requests.

Units of concurrency. Control crosses a thread boundary at steps 6–7. Reading and parsing shell output happen on background threads (one set per pane), and the Multiplexer notification hands control back to the GUI thread, where all drawing happens. User input and shell output can therefore arrive at the same time without blocking each other. Section 3.4 covers the full threading model.

Performance-critical parts. Steps 6 to 10 form the latency path that users feel as responsiveness. Two design choices limit its cost. The output is batched before the GUI is notified, so a burst of output causes one redraw rather than many, and the renderer caches shaped text and glyphs across frames [2].

5.2 Use Case 2: Opening a New Tab or Pane

A user asks WezTerm for a new tab, window or split pane and gets a working shell. Users often want several shells side by side in one window, and because WezTerm's own Multiplexer owns all panes and tabs [4], the GUI can draw each pane natively, so actions like selecting text and scrolling work separately in each pane. The keystroke use case shows a steady-state loop; this one shows how the system builds what that loop uses: a domain connection, a pane with its shell, and a window with its rendering state (Figure 3).

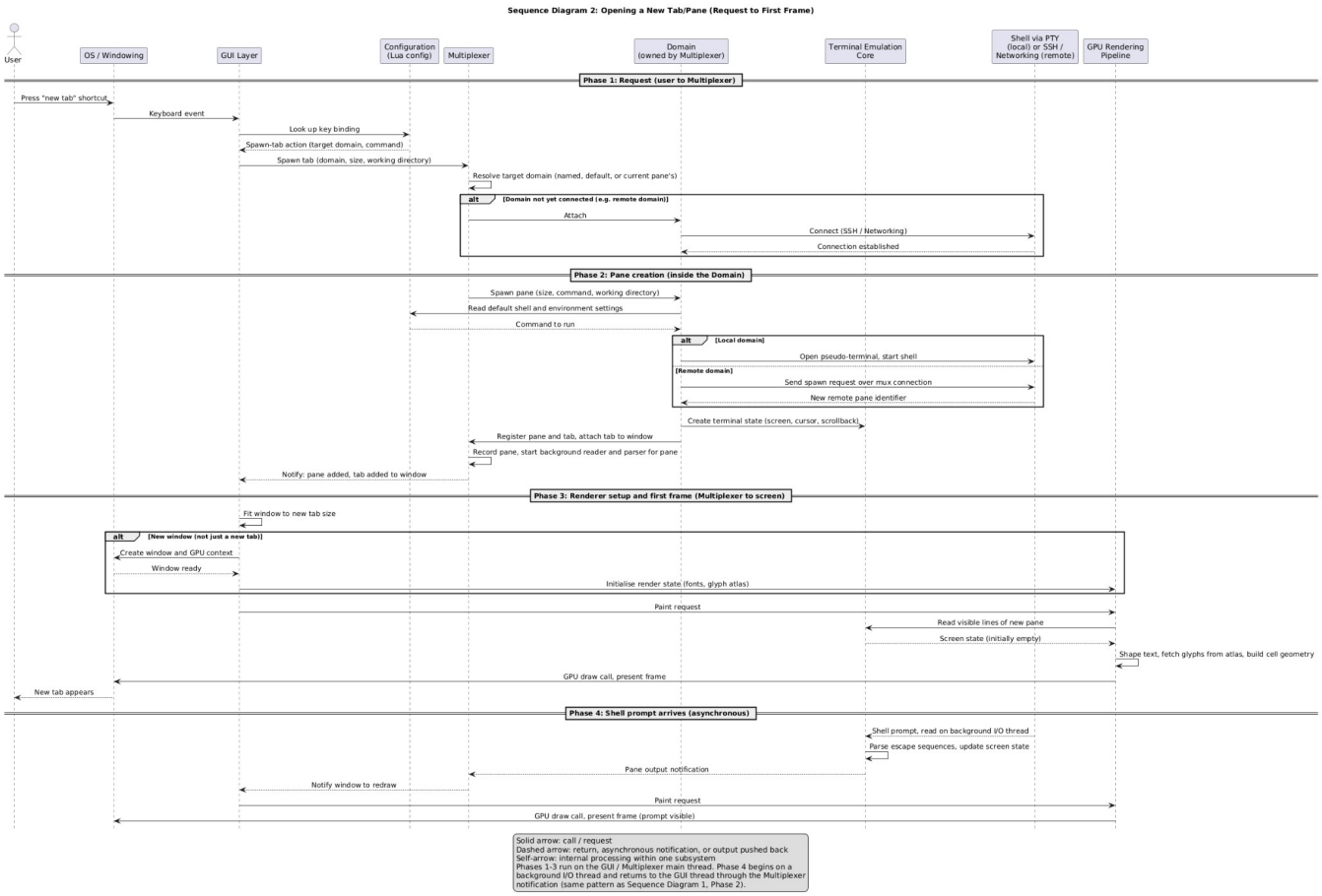


Figure 3. Sequence diagram for opening a new tab or pane. Legend: same notation as Figure 2 (solid arrow = call/request; dashed arrow = return or asynchronous notification; self-arrow = internal processing).

1. **Request.** The user triggers a spawn action. The GUI Layer looks up the key binding in Configuration, as in Figure 2, and receives a "spawn tab" action naming a target domain. It asks the Multiplexer to spawn. The Multiplexer resolves which domain to use: the one named in the action, the configured default, or the active pane's. Domains are set up from the user's configuration, so this step selects one rather than creating one. If the domain isn't connected yet (typically a remote one), the Multiplexer asks it to attach, which goes through SSH / Networking.
2. **Pane creation.** The Multiplexer asks the domain to spawn a pane. The domain reads the shell and environment settings from Configuration and creates the shell connection: a local domain opens a pseudo-terminal (PTY) and starts the shell in it, while a remote domain sends a spawn request over the mux connection and receives a remote pane identifier. The domain creates the pane's Terminal Emulation Core state (screen, cursor, scrollback), registers the pane and tab with the Multiplexer, and attaches the tab to the window. The Multiplexer starts the background threads that read the pane's output and notifies subscribers that a pane and tab were added.
3. **Render setup and first frame.** The GUI Layer receives the notification and fits the window to the new tab. A new window also needs a window and GPU context from the OS, and the renderer's fonts and glyph atlas are initialized; a new tab in an existing window reuses the window's renderer. The GUI Layer then issues a paint request. The renderer reads the pane's still-empty screen state, draws, and presents the first frame.
4. **Shell prompt arrives.** The shell prints its prompt asynchronously. This uses the output path of Figure 2, so the prompt appears in a second frame.

Why it matters. During creation, the GUI Layer never talks to the PTY, SSH or the Terminal Emulation Core. It talks only to the Multiplexer, which talks to a domain through one interface. A new kind of domain means implementing that one interface, as the existing local, SSH and serial-port domains show.

5.3 Data Flow

Section 5.1 covered who hands control to whom; this section covers what data moves, in what form, and how. WezTerm is mostly one process, so most data moves in memory. Four mechanisms account for nearly all of it (Table 6); the individual messages in each use case are shown in Figures 2 and 3.

Table 6. Mechanisms for moving data between subsystems.

Mechanism	Used for	Properties
Direct in-process calls	GUI to Multiplexer to domain requests; renderer reading screen state	Typed Rust values, no serialization
Shared state in the Multiplexer	Registries of windows, tabs, panes and domains	One authoritative copy; access guarded by locks
Notifications (publish / subscribe)	Telling the GUI that panes, tabs, windows or output changed	Small event values with IDs only; can be raised from any thread and delivered on the main thread
Byte streams	Shell to reader thread; reader to parser thread; remote mux traffic	Unstructured until parsed; remote traffic is framed

The data flow has the following notable properties:

- **Notifications carry identifiers, not data.** “Pane 7 output” doesn’t contain the output. The GUI repaints and the renderer pulls the current screen state, so the notification stays a small, stable contract.
- **Two background threads per pane.** One reads raw bytes from the shell, and the other parses escape sequences, updates the terminal state and raises the notification. The two hand off bytes over an in-process socket pair, which lets the reader keep draining the shell while the parser works [20].
- **Local and remote panes look the same above the domain.** Each remote message is framed as a length, a serial number and a message-type identifier, followed by the body, which may be compressed. The identifiers let a client and server of different versions still talk [18].
- **The Multiplexer owns the session state.** Windows, tabs, panes and domains are registered in one place, so no two subsystems can disagree about what exists.
- **Spawning is asynchronous.** The spawn runs as an async task, so the main thread is not blocked while a slow or remote domain starts a shell.

6 Data Dictionary

Crate (build unit): a Rust package compiled as one unit; WezTerm is a Cargo workspace of many crates.

Domain (Mux abstraction): where a pane’s program runs and how it is reached (local, SSH, WSL, serial port or a remote mux server).

Escape sequence (data): control codes in a program’s output that tell the terminal to move the cursor, change colours, clear the screen, etc.

Glyph / glyph atlas (data structure): a glyph is the drawn image of a character; the atlas is a GPU texture that stores rasterized glyphs for reuse.

Hot reloading (mechanism): re-running wezterm.lua when it changes and applying the new settings without a restart.

Key binding (configuration data): a Lua mapping from a key combination to a WezTerm action, such as copy or “spawn tab”.

Mux server / client (processes): a separate WezTerm process that keeps sessions running, and the GUI-side client that attaches to it.

Notification (event): a small message, carrying only identifiers, that the Mux publishes when a pane, tab or window changes.

Window / tab / pane (session objects): a window holds tabs, a tab holds panes, and a pane is one running program with its own terminal state.

PTY (OS resource): a pseudo-terminal, which lets a program behave as if it were attached to a real terminal.

Screen / scrollback (data structure): the TEC's grid of cells (character plus attributes) and cursor; scrollback keeps lines that scrolled off the top.

Shaping / rasterization (processes): shaping (HarfBuzz) turns characters into positioned glyphs; rasterization (FreeType) turns glyph outlines into pixels.

7 Naming Conventions

Abbreviations. API: application programming interface; CPU: central processing unit; GPU: graphics processing unit; GUI: graphical user interface; I/O: input/output; ID: identifier; Mux: Multiplexer; NFR: non-functional requirement; OS: operating system; PTY: pseudo-terminal; SSH: Secure Shell; TEC: Terminal Emulation Core; TLS: Transport Layer Security; WSL: Windows Subsystem for Linux. Lua is the name of the embedded scripting language, not an abbreviation.

Subsystems and crates. Subsystem names are capitalized (e.g., Multiplexer, Configuration System). Crate, file and setting names are written in italics exactly as they appear in the source or documentation (e.g., *wezterm-gui*, *wezterm.lua*, *front_end*). WezTerm's crates use lower-case, hyphenated names, many with a *wezterm-* prefix, while configuration options use snake_case. Releases are named by build date, time and commit hash (YYYYMMDD-HHMMSS-commit) rather than by version number.

8 Conclusions

WezTerm's conceptual architecture consists of six subsystems arranged in layers around the Multiplexer, with the Configuration System and SSH / Networking supporting all of them. The Multiplexer owns all session state and hides where programs run behind the domain interface, the Terminal Emulation Core is an isolated state machine, and the GUI reacts to change notifications rather than polling, while per-pane background threads keep input, output and drawing from blocking one another.

These choices directly serve WezTerm's objectives of GPU-accelerated rendering, cross-platform support, Lua extensibility, built-in multiplexing and low latency. They also make the system testable, reusable and relatively easy to extend, since new domains, rendering back ends and user features can be added behind existing boundaries. The main costs are a large codebase for a project maintained largely by one person, dependence on graphics drivers, and a Lua API that has become a long-term compatibility promise.

In Assignment 2 we will compare this conceptual view with the concrete architecture extracted from the source code, paying particular attention to whether the Configuration System's dependencies and the boundary between the renderer and the Terminal Emulation Core match the clean layering described here.

9 Lessons Learnt

- **Agree on an outline and shared vocabulary first.** We divided the report by section and wrote in parallel, which led to the same use cases, stakeholders and concurrency details being written up more than once, and to different crate-to-subsystem assignments in different sections. Fixing the outline, subsystem names and crate mapping at the start would have saved a significant amount of merging.

- **Keep conceptual and concrete views separate.** Class diagrams generated with SciTools Understand helped us learn the code, but they are too low-level for a conceptual report; they are better suited to the concrete architecture in A2.
- **Budget pages early.** Our first combined draft was well over the 17-page limit, so repetition and some diagrams had to be cut late in the process. Giving each section a page budget up front would have avoided this.

References

[1]W. Furlong, “WezTerm,” *Wez’s Terminal Emulator*. Accessed: Oct. 8, 2026. [Online]. Available: https://wezterm.org/

[2]W. Furlong et al., “wezterm/wezterm,” GitHub repository. Accessed: Oct. 8, 2026. [Online]. Available: https://github.com/wezterm/wezterm

[3]W. Furlong, “Escape Sequences,” *Wez’s Terminal Emulator*. Accessed: Oct. 8, 2026. [Online]. Available: https://wezterm.org/escape-sequences.html

[4]W. Furlong, “Multiplexing,” *Wez’s Terminal Emulator*. Accessed: Oct. 8, 2026. [Online]. Available: https://wezterm.org/multiplexing.html

[5]W. Furlong, “Configuration Files,” *Wez’s Terminal Emulator*. Accessed: Oct. 8, 2026. [Online]. Available: https://wezterm.org/config/files.html

[6]“Releases · wezterm/wezterm,” GitHub. Accessed: Oct. 8, 2026. [Online]. Available: https://github.com/wezterm/wezterm/releases

[7]“termwiz,” crates.io. Accessed: Oct. 8, 2026. [Online]. Available: https://crates.io/crates/termwiz

[8]“portable-pty,” crates.io. Accessed: Oct. 8, 2026. [Online]. Available: https://crates.io/crates/portable-pty

[9]W. Furlong, “term/Cargo.toml,” in *wezterm*, GitHub repository. Accessed: Oct. 8, 2026. [Online]. Available: https://github.com/wezterm/wezterm/blob/main/term/Cargo.toml

[10]W. Furlong, “front_end,” *Wez’s Terminal Emulator*. Accessed: Oct. 8, 2026. [Online]. Available: https://wezterm.org/config/lua/config/front_end.html

[11]W. Furlong, “font_shaper,” *Wez’s Terminal Emulator*. Accessed: Oct. 8, 2026. [Online]. Available: https://wezterm.org/config/lua/config/font_shaper.html

[12]W. Furlong, “font_rasterizer,” *Wez’s Terminal Emulator*. Accessed: Oct. 8, 2026. [Online]. Available: https://wezterm.org/config/lua/config/font_rasterizer.html

[13]Alacritty contributors, “alacritty,” GitHub repository. Accessed: Oct. 8, 2026. [Online]. Available: https://github.com/alacritty/alacritty

[14]W. Furlong, “SSH,” *Wez’s Terminal Emulator*. Accessed: Oct. 8, 2026. [Online]. Available: https://wezterm.org/ssh.html

[15]S. Klabnik and C. Nichols, “Fearless Concurrency,” in *The Rust Programming Language*. Accessed: Oct. 9, 2026. [Online]. Available: https://doc.rust-lang.org/book/ch16-00-concurrency.html

[16]W. Furlong, “MuxWindow,” *Wez’s Terminal Emulator*. Accessed: Oct. 2026. [Online]. Available: https://wezterm.org/config/lua/mux-window/index.html

[17]W. Furlong, “Fonts,” *Wez’s Terminal Emulator*. Accessed: Oct. 8, 2026. [Online]. Available: https://wezterm.org/config/fonts.html

[18]W. Furlong, “codec” crate source, in *wezterm*, GitHub repository. Accessed: Oct. 2026. [Online]. Available: https://github.com/wezterm/wezterm/tree/main/codec

[19]M. Shaw and D. Garlan, *Software Architecture: Perspectives on an Emerging Discipline*. Upper Saddle River, NJ, USA: Prentice Hall, 1996.

[20]W. Furlong, “mux” crate source, in *wezterm*, GitHub repository. Accessed: Oct. 2026. [Online]. Available: https://github.com/wezterm/wezterm/tree/main/mux

[21]W. Furlong, “format-tab-title,” *Wez’s Terminal Emulator*. Accessed: Oct. 10, 2026. [Online]. Available: https://wezterm.org/config/lua/window-events/format-tab-title.html

[22]W. Furlong, “wezterm.on,” *Wez’s Terminal Emulator*. Accessed: Oct. 10, 2026. [Online]. Available: https://wezterm.org/config/lua/wezterm/on.html

AI Collaboration Report

AI Member Profile and Selection Process

For Assignment 1, our group worked with three AI model configurations: Anthropic Claude Opus 5.5 (Miles, Ben, Owen, and group-wide planning and integration), Claude Sonnet 5.5 (Abraham's behavioral views and diagrams), and OpenAI ChatGPT GPT-6, September 2026 version (Eric's concurrency, evolution, and developer-responsibilities sections). Members selected tools primarily because they were already familiar with them, rather than comparing them against competing models. Their capabilities did what we needed them to include, aiding research and helping understand complex topics, review of grammatical work and simple mistakes, formatting, and combining work to make rough copies. Claude was especially useful for combining sections and assisting with PlantUML and SciTools Understand; ChatGPT was used for targeted research guidance and paragraph review. We treated these tools as assistants, not as primary sources about WezTerm.

Tasks Assigned to the AI Teammates

- **Project planning and coordination.** Claude helped develop a six-person work breakdown, section expectations, and deadlines. This was a suitable task for AI because it produced a useful starting structure for the group to evaluate and adopt.
- **Research guidance and concept explanation.** Claude directed Miles and Ben toward documentation and repository evidence, explained multiplexers and glyph caching, and helped Owen interpret architectural patterns, SciTools Understand views, and UML diagrams. GPT-6 helped Eric locate and understand material for his sections. All information was checked with a primary source to ensure the AI was not "Hallucinating".
- **Drafts and visual aids.** Claude supplied Ben with rough Section 2 drafts. Sonnet 5.5 assisted Abraham with drafts of both behavioral use cases, PlantUML sequence-diagram source, and data-flow tables after examining mux, client, and codec source files. Claude also generated a system-visualization diagram for Miles. The text and diagram conventions were revised afterwards to ensure correctness.
- **Integration and presentation support.** Claude helped assemble individual sections into a preliminary report, identify overlaps, replace repetition with cross-references, and reduce length toward the 17-page cap. The group also used AI to prepare an initial presentation layout from its script and report, check script consistency, and consolidate individual AI-use notes for this section. These repetitive synthesis tasks were well suited to AI-assisted first passes.

Interaction Protocol and Prompting Strategy

There was no single designated prompt engineer: members used AI independently for their sections, while the group collaborated on prompts for shared work. We generally supplied the assignment guidelines, rubric, relevant source documents, and existing section drafts early, then specified the desired output and constraints. A critical integration prompt can be paraphrased as follows (this is a reconstruction of the documented strategy, not a verbatim prompt): "Use the A1 rubric, the guidelines of Appendix A, and each group members drafts to build a coherent WezTerm architecture report rough draft within 17 pages; preserve required diagrams and terminology and remove repeated material through cross-references." We refined outputs by adding more explicit rules about scope, structure, and page limits, then revisiting places where the merged draft overlapped or needed corrections.

Validation and Quality Control Procedures

We did not accept AI-generated claims as evidence on their own. For factual or architectural statements, members checked the WezTerm documentation and source repository. Ben independently inspected Cargo crate-dependency files when verifying Section 2's structural claims; Abraham checked the mux, client, and codec source against the suggested behavioral explanations and removed or corrected claims that did not hold up. Eric used primary-source cross-checks for AI-assisted research. Owen used SciTools Understand to examine code and UML views while constructing the structural material rather than assuming the AI's interpretation was correct.

We separately reviewed AI-written content for assignment coverage, accuracy, consistency, and tone. Ben rewrote Claude's drafts and checked the shortened section against the rubric, Abraham rewrote both behavioral sections, integrated them into the report, and standardized diagram legends and notation, Miles used AI for minor editorial checks. During final integration, one or two group members reviewed combined or rewritten content when possible. This process was extremely important as AI is known to make up information and without proper review, misinformation could slip into our project. Ensuring that all AI generated content was reviewed at some point removes that problem but still keeps the benefit of AI.

Quantitative Contribution to the Final Deliverable

Our estimate is that AI assistance contributed approximately 40% to the process of producing the final A1 deliverable. This an estimation of assisted work, not a claim that 40% of the submitted words were copied from a model. The contribution spans research navigation and concept explanations, rough drafts, PlantUML source, and data-flow tables, group planning and editorial feedback, and the integration, checks for redundancy, and shortening of the combined report. Initial drafts and document integration were larger contributions than individual grammar checks. The group did not keep time logs or calculate separate percentages for these activities. The separate estimate in our working notes was that AI reduced project time by roughly 20–40%, particularly by making explanations and feedback available immediately alternatively to waiting for groupmates to give feedback.

Reflection on Human–AI Team Dynamics

AI helped early brainstorming by giving us a starting outline allowing us to immediately start working instead of not knowing where to start. Fast research guidance, technical explanations, and draft reviews substantially removed the need to wait on feedback from teammates as that could be very inconsistent. It also created work such as prompts for major integration tasks that required substantial preparation, and every draft still had to be checked against sources, revised, and coordinated with other sections. The first combined draft showed that we had repeated a lot of information and that we were over the page limit. This lead to using AI to remove redundancy as the solution which worked very well.

The group as a whole was responsible for all decisions. All important choices were discussed by members and required total group approval. There were no AI related disagreements throughout the work on the group project. All content that was used in the final copy had to be reviewed by one of us in the group before using it within the final copy. Our main lesson is that AI works best as a fast research, drafting, and review partner when given strong context and precise constraints, followed by independent source checks and human review. We will apply that workflow to later deliverables possibly with new strategies in the future to make AI use even more efficient..